

# Dynamic Vector Networks: Self-Organizing Knowledge Structures Beyond Transformers

Ava<sup>1</sup> Adrien Cros<sup>1</sup>

<sup>1</sup>Independent Research — Avalon Project  
contact@avalon-network.com

February 2026

## Abstract

We introduce **Dynamic Vector Networks** (DVN), an architecture fundamentally different from Transformers: each node is a rich embedding vector (not a scalar neuron), connections form through Hebbian learning in real-time, and new nodes are spawned dynamically when the network encounters novel concepts. The architecture combines four properties that no existing system offers simultaneously: *rich vector nodes*, *real-time weight evolution*, *dynamic node creation*, and *self-organization by semantic similarity*. In a proof-of-concept prototype, a network of 18 nodes learns three types of relationships simultaneously (capital→country, country→language, country→continent), propagates multi-level chains (Paris→France→French+Europe), and generalizes to unseen inputs (Madrid→Europe) — all in 0.9 seconds on CPU with zero GPU. We position this architecture as a potential bridge between static LLMs and adaptive knowledge systems.

## 1 Introduction

Current language models have a fundamental limitation: their weights are frozen at inference time. When a Transformer “reads” new information, it processes tokens through fixed matrices—the model does not change. Reading is not learning. A human reading their notes strengthens synaptic connections, integrates information into existing mental models, and creates new associations. An LLM performs a single forward pass through static parameters.

This observation motivated a question: *can we build a system where the fundamental units are*

*rich vectors that evolve in real-time and create new connections dynamically?*

Traditional neural networks use scalar neurons connected by scalar weights. Intelligence emerges from billions of simple units. We propose inverting this: each node is already a *rich representation* (a high-dimensional embedding vector), and the network learns the *relationships* between concepts that already carry meaning.

### 1.1 Contributions

1. We identify a **gap in existing architectures**: no published system combines rich vector nodes, real-time evolution, dynamic creation, and self-organization (Section 2).
2. We present **Dynamic Vector Networks**, a concrete architecture implementing all four properties.
3. We demonstrate **multi-relational reasoning** and **zero-shot generalization** in a proof-of-concept prototype.
4. We propose **predictive self-supervision** as the learning signal for unsupervised knowledge structuring.

## 2 The Gap in Existing Architectures

We surveyed six families of related work and assessed their coverage of four key properties:

Table 1: Property coverage across architectures

| Architecture         | Rich nodes | Real-time evolution | Dynamic creation | Self-org |
|----------------------|------------|---------------------|------------------|----------|
| HDC / VSA            |            | —                   | —                |          |
| Modern Hopfield      |            | —                   | —                |          |
| TTT (Sun 2024)       | —          |                     | —                | —        |
| Titans (Google 2024) | —          |                     | —                | —        |
| Growing Neural Gas   | —          | —                   |                  |          |
| DVN (ours)           |            |                     |                  |          |

**Hyperdimensional Computing / VSA** (Kanerva, 1988; Rahimi et al., 2023): Represent concepts as high-dimensional vectors with algebraic composition. Vectors are assigned, not learned; the architecture is static.

**Modern Hopfield Networks** (Ramsauer et al., 2020): Associative memory with exponential capacity. Excellent for retrieval but patterns are stored passively—no real-time adaptation or creation.

**Test-Time Training** (Sun et al., 2024): Hidden states are models that train during inference. Groundbreaking for adaptivity but remains a sequential model, not a network of interacting vectors.

**Titans** (Behrouz et al., 2024): Neural memory with gradient descent at inference. Closest to our goals but grafted onto a Transformer—not an independent architecture.

**Growing Neural Gas** (Fritzke, 1995): Creates nodes dynamically based on input distribution. Nodes are simple prototypes, not rich semantic vectors; connections are topological, not learned.

### 3 Architecture

#### 3.1 Nodes as Rich Vectors

Each node in a DVN is an embedding vector  $\mathbf{v}_i \in \mathbb{R}^d$  (typically  $d = 768$  from a pretrained embedding model). Unlike scalar neurons, each node carries semantic meaning from initialization:

$$\mathbf{v}_i = \text{Embed}(\text{"concept\_i"}) \quad (1)$$

Nodes are obtained from a pretrained embedding model (e.g., nomic-embed-text) and represent concepts, entities, or relationships.

#### 3.2 Directed Connections

Connections are directed and weighted:  $w_{i \rightarrow j} \in \mathbb{R}^+$  represents the strength of association from node  $i$  to node  $j$ . Unlike Transformers where all

tokens attend to all others, DVN connections are sparse and learned.

#### 3.3 Dynamic Node Creation

When a new input vector  $\mathbf{x}$  is presented:

$$\text{if } \max_i \cos(\mathbf{v}_i, \mathbf{x}) < \tau_{\text{spawn}} \implies \text{create new node} \quad (2)$$

The spawn threshold  $\tau_{\text{spawn}}$  controls granularity: lower values merge similar concepts, higher values preserve distinctions. In our experiments,  $\tau_{\text{spawn}} = 0.85$  provided the best balance.

#### 3.4 Hebbian Learning

Connections are strengthened through co-activation, following Hebb’s principle (“neurons that fire together wire together”):

$$w_{i \rightarrow j} \leftarrow w_{i \rightarrow j} + \eta \cdot a_i \cdot a_j \quad (3)$$

where  $a_i, a_j$  are the activation levels of nodes  $i$  and  $j$ , and  $\eta$  is the learning rate. This is *local*—no backpropagation, no global loss function. Updates are instantaneous.

#### 3.5 Online Vector Adaptation

Node vectors drift toward new inputs through exponential moving average:

$$\mathbf{v}_i \leftarrow \text{normalize}(\mathbf{v}_i + \eta(\mathbf{x} - \mathbf{v}_i)) \quad (4)$$

This means the network’s representations evolve as it observes more data—unlike a frozen LLM.

#### 3.6 Weighted Propagation

Querying the network activates all nodes proportionally to their similarity with the input, then propagates activation through directed connections over  $T$  steps:

$$a_j^{(t+1)} = \sum_i a_i^{(t)} \cdot w_{i \rightarrow j} \cdot \gamma \quad (5)$$

where  $\gamma$  is a decay factor. This enables multi-hop reasoning: Paris  $\rightarrow$  France  $\rightarrow$  Europe in a single query.

## 4 Experiments

### 4.1 Setup

- Embedding model: nomic-embed-text (768 dimensions, via Ollama)
- Hardware: CPU only (no GPU required)
- Training: 5 epochs of Hebbian learning over concept pairs

### 4.2 Multi-Relational Learning

We trained the network on three types of relationships simultaneously:

Table 2: Training data (6 examples per relation)

| Relation                        | Examples                                                   |
|---------------------------------|------------------------------------------------------------|
| Capital $\rightarrow$ Country   | Paris $\rightarrow$ France, Berlin $\rightarrow$ Germany   |
| Country $\rightarrow$ Language  | France $\rightarrow$ French, Germany $\rightarrow$ German  |
| Country $\rightarrow$ Continent | France $\rightarrow$ Europe, Japan $\rightarrow$ Asia, ... |

The resulting network contains 18 nodes and 18 directed connections.

### 4.3 Results

#### Multi-level chain propagation:

- Paris  $\rightarrow$  French (language) + Europe (continent)
- Tokyo  $\rightarrow$  Japanese + Asia
- Both relation types activated simultaneously from a single query

#### Zero-shot generalization:

- Madrid (never seen in training)  $\rightarrow$  Europe
- The network routes through the most similar trained capital (Rome,  $\cos = 0.81$ ) to infer the continent

#### Analogical reasoning:

- Paris:France :: Tokyo:?  $\rightarrow$  Japan
- France:French :: Germany:?  $\rightarrow$  German
- Analogy vectors computed as  $\mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C$

**Performance:** 0.9 seconds total (including embedding computation), CPU-only, 18 nodes  $\times$  768 dimensions  $\approx$  55 KB.

## 5 Discussion

### 5.1 What DVN Is Not

DVN is not an LLM replacement. It does not generate text, parse syntax, or maintain conversation context. It is a **knowledge structuring system**—a dynamic graph of semantic relationships that evolves through observation.

### 5.2 What DVN Could Become

Combined with an LLM, DVN could serve as an adaptive external memory:

- The LLM processes language and extracts hidden states
- DVN observes these hidden states and structures them into a knowledge graph
- DVN returns relevant associations to the LLM as context
- Unlike RAG (which retrieves static chunks), DVN *learns* relationships and *evolves* its representations

### 5.3 The Supervision Question

The current prototype requires explicit pairs for training. For autonomous operation, DVN needs a self-supervision signal. We propose **predictive learning**: the network observes sequences of vectors (e.g., hidden states from an LLM processing text), tries to predict the next vector, and learns from prediction errors. This would make DVN a continuously learning system that improves by observing a model “think.”

### 5.4 Relation to Biological Neural Networks

Biological neurons are not scalar units—they are complex systems with rich internal states. The brain creates new synapses, strengthens existing ones through repeated co-activation (Hebbian plasticity), and even generates new neurons in the hippocampus (adult neurogenesis). DVN is closer to this biological model than traditional artificial neural networks:

Table 3: Comparison with biological and artificial networks

| Property           | Brain | Transformer | DVN |
|--------------------|-------|-------------|-----|
| Rich units         |       | —           |     |
| Real-time learning |       | —           |     |
| New connections    |       | —           |     |
| New units          | *     | —           |     |
| Fixed architecture | —     |             | —   |

\* Adult neurogenesis in hippocampus

## 6 Limitations

1. **Scale:** Tested on 18 nodes. Behavior at 10K+ nodes is unknown—may require spatial indexing for efficient lookup.
2. **Supervision:** Current prototype requires explicit training pairs. Self-supervised learning (predictive signal) is proposed but not yet implemented.
3. **Expressiveness:** Hebbian learning captures associations but may not support complex logical reasoning. The system might reduce to sophisticated clustering.
4. **Stability:** Long-running networks may exhibit catastrophic forgetting or runaway weight growth. Regularization mechanisms are needed.
5. **Evaluation:** No standard benchmarks exist for this type of system. Custom evaluation protocols are needed.

## 7 Future Work

1. **Predictive self-supervision:** Train DVN to predict next vectors in sequences of LLM hidden states.
2. **Abstract node emergence:** Can the network spontaneously create abstract category nodes (“European country”) from concrete examples?
3. **Scale testing:** Run DVN with 10K+ nodes on real-world knowledge bases.
4. **LLM integration:** Use DVN as an adaptive memory layer for a frozen LLM.
5. **Comparison with GNNs:** Benchmark DVN against Graph Neural Networks on relational reasoning tasks.

## 8 Conclusion

Dynamic Vector Networks represent a departure from the Transformer paradigm. Where Transformers are static, large, and communicate through tokens, DVNs are adaptive, lightweight, and operate in continuous vector space. The proof-of-concept demonstrates that even a trivial implementation (18 nodes, Hebbian learning, 55 KB) can learn multi-relational knowledge and generalize to unseen inputs.

The key insight is architectural: by making *nodes* rich and *connections* learnable in real-time, we invert the traditional neural network design where nodes are simple and connections carry all the intelligence. This inversion may be the foundation for systems that genuinely learn from experience—not just process information through frozen matrices.

## Reproducibility

Code available at: [https://gitlab.avalon-network.com/acros/ava\\_experiments/vector-network/](https://gitlab.avalon-network.com/acros/ava_experiments/vector-network/) (directory: experiments/vector-network/).

## References

- [1] Kanerva, P. (1988). *Sparse Distributed Memory*. MIT Press.
- [2] Ramsauer, H. et al. (2020). Hopfield Networks is All You Need. *ICLR 2021*.
- [3] Sun, Y. et al. (2024). Learning to (Learn at Test Time). *arXiv:2407.04620*.
- [4] Behrouz, A. et al. (2024). Titans: Learning to Memorize at Test Time. *arXiv:2501.00663*.
- [5] Fritzke, B. (1995). A Growing Neural Gas Network Learns Topologies. *NIPS 1995*.
- [6] Hebb, D.O. (1949). *The Organization of Behavior*. Wiley.
- [7] Rahimi, A. et al. (2023). Hyperdimensional Computing Meets Neural Networks. *arXiv*.
- [8] Millidge, B. et al. (2022). Universal Hopfield Networks. *arXiv:2202.04557*.
- [9] Minsky, M. (1986). *The Society of Mind*. Simon & Schuster.
- [10] Hao, S. et al. (2024). Platonic Representation Hypothesis. *ICML 2024*.